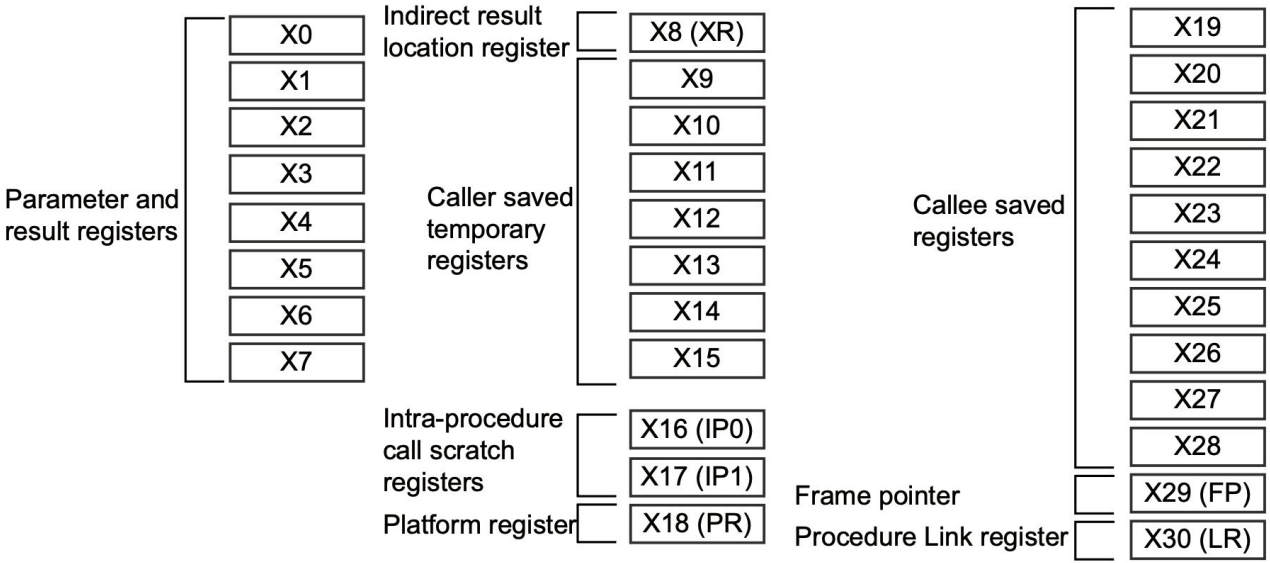
Questions

- Why do we not have AArch128?
- Why are instructions 64 bits wide?
- How to change exception levels and can we go to EL3 somehow?
- Why are actual addresses of memory use < 64 bits? Why are spaces reserved?
-

The AAPCS64 Contract

- **The Application Binary Interface (ABI):** The contract that dictates how C, Rust, and Assembly share the physical CPU registers without destroying each other's data.



The AAPCS64 Contract (Caller-Saved Registers)

- **Caller-Saved (Volatile) Registers:** If Function A calls Function B, Function B is allowed to overwrite these registers. If Function A needs this data later, Function A must push it to RAM (the Stack) before making the call.
- **X0 - X7 (Arguments & Results):**
  - Used to pass the first 8 arguments to a function.
  - X0 (and sometimes X1) holds the return value.
- **X9 - X15 (Temporary Scratch):**
  - Functions can use these freely without worrying about restoring them.

The AAPCS64 Contract (Callee-Saved Registers)

- **Callee-Saved (Non-Volatile) Registers:**
  - If Function B wants to use these registers, Function B must back them up to the Stack first, and restore them to their exact original state before returning to Function A.
- **X19 - X28 (Local Variables):**
  - Used for long-lived data that must survive across multiple nested function calls.
- **Why the split? (Hardware Optimization):**
  - Saving and restoring all 31 registers to RAM on every function call would destroy memory bandwidth. The ABI split minimizes RAM access.

Special Purpose AAPCS Registers

- **X8 (Indirect Result Location):**
  - How do you return a 100-byte struct if a register is only 8 bytes? You cannot! The caller allocates space in RAM and passes a hidden pointer to that space via X8.
- **X16 & X17 (IP0 / IP1 - Intra-Procedure Call):**
  - Reserved for the Linker. Used to insert "Veneers" (trampolines) when a function call (BL) exceeds the maximum physical jump range (±128 MB).
  - *Veneers are small pieces of code which are automatically inserted by the linker.*

Special Purpose AAPCS Registers

- **X18 (Platform Register):**
  - Its specific use is not defined by the hardware, but by the software platform (the Operating System) running on it.
- **X29 (Frame Pointer - FP):**
  - Points to the base of the current function's memory frame on the Stack. Mandatory for debuggers to trace the call history (Backtracing).
- **X30 (Link Register - LR):**
  - Stores the return address (PC + 4) of the caller.

Data Processing

The ALU

- Syntax: MNEMONIC Destination, Source1, Source2
- Standard Arithmetic:
  - **ADD X0, X1, X2** → X0 = X1 + X2
  - **SUB W0, W1, #10** → W0 = W1 - 10
  - **MUL X0, X1, X2** → X0 = X1 * X2 (Calculates lower 64 bits only).
- Because every instruction is strictly 32 bits long, you cannot embed a massive 64-bit constant directly into an ADD or SUB instruction.
- Standard arithmetic instructions only accept a 12-bit immediate (0 to 4095).

The Inline Barrel Shifter

- The ALU is wired to a hardware shifter. The **second operand** can be shifted or rotated as it enters the ALU, consuming zero extra clock cycles.
- **LSL** (Logical Shift Left): Multiplies by 2^n. Fills with 0s.
- **LSR** (Logical Shift Right): Unsigned divide by 2^n. Fills with 0s.
- **ASR** (Arithmetic Shift Right): Signed divide. Preserves the sign bit.
- Example:
  - **ADD X0, X1, X2, LSL #3** → X0 = X1 + (X2 * 8)

Bitwise Logic & Hardware Control

- Logical Operations: Process data bit-by-bit.
  - **AND X0, X1, #0xFF** (Masking: Isolates the lowest byte).
  - **ORR X0, X1, #0x08** (Setting: Forces bit 3 to 1).
  - **EOR X0, X1, X2** (XOR: Toggles bits).
- The BIC Instruction (Bit Clear):
  - Performs a bitwise AND NOT.
  - **BIC X0, X1, #0x0F** (Forces the lowest 4 bits to 0, leaves the rest untouched).
- Data Reversal:
  - **REV X0, X1**: Reverses byte order (Endianness swapping).

Memory Access & Addressing Modes

The Load/Store Foundation

- The 64-bit Access (Doubleword):
  - **LDR X0, [X1]** (Reads 8 bytes from the address in X1 into X0).
  - **STR X0, [X1]** (Writes 8 bytes).
- The 32-bit Access (Word):
  - **LDR W0, [X1]** (Reads 4 bytes). Upper 32 bits of X0 are zeroed.
- The Sub-Word Access:
  - **LDRH W0, [X1]** (Halfword: 2 bytes).
  - **LDRB W0, [X1]** (Byte: 1 byte).

The Sign-Extension Trap

- **The Problem:** The ALU only performs 32-bit or 64-bit mathematics. It cannot do 8-bit math. Sub-word data must be promoted to register width.
- Zero-Extension (Default):
  - **LDRB W0, [X1]** loads the byte **0xFF**.
  - Result in **W0**: **0x000000FF** (+255 decimal).
- Sign-Extension (Explicit):
  - If 0xFF represents -1 (Two's Complement), default loading corrupts the math.
  - **LDRSB W0, [X1]** (Load Register Signed Byte).
  - Result in **W0**: **0xFFFFFFFF** (-1 decimal).

Addressing Modes & Hardware Iteration

- **Base Register**: **LDR X0, [X1]** (Access exact address in X1).
- **Base + Offset**: **LDR X0, [X1, #8]** (Access X1 + 8 bytes). X1 remains unchanged.
- **Pre-Indexed (Writeback)**: **STR X0, [X1, #8]!**
  - ! explicitly commands the CPU to update the base pointer.
  - X1 = X1 + 8
  - Store X0 at the new address in X1.
- **Post-Indexed (Array Traversal)**: **LDR X0, [X1], #8**
  - Offset is outside the brackets.
  - Load data from the original address in X1 into X0.
  - X1 = X1 + 8.

Load/Store Pairs (LDP / STP)

- **LDP** (Load Pair) and **STP** (Store Pair).
- Moves exactly two registers at a time.
- **STP X29, X30, [SP, #-16]!** (Pushes Frame Pointer and Link Register to the stack, updating the Stack Pointer).
- **LDP X29, X30, [SP], #16** (Pops them off the stack).

Control Flow & PSTATE

PSTATE and the Condition Flags (NZCV)

- **The Problem:** The CPU ALU does not understand logical concepts like ">" or "==".
- **The Solution:** A dedicated internal system register (Process State) containing four physical electrical flags (bits) updated by the ALU.
- The NZCV Flags:
  - **N** (Negative): Set to 1 if the result of the last operation was negative (specifically, if the Most Significant Bit [bit 31 or 63] is 1).
  - **Z** (Zero): Set to 1 if the result of the last operation was exactly zero.
  - **C** (Carry): Set to 1 if an unsigned overflow occurred (a carry out of the MSB).
  - **V** (oVerflow): Set to 1 if a signed overflow occurred.

Setting the Flags (The 'S' Suffix)

- Standard arithmetic (**ADD**, **SUB**) **does not** update the PSTATE flags.
- You must explicitly command the ALU to update the flags by appending an **S**.
- **SUBS X0, X1, X2** (Subtract, store result in **X0**, update **NZCV**).
- **ADDS X0, X1, X2** (Add, store result in **X0**, update **NZCV**).
- The CMP (Compare) Alias:
  - **CMP X1, X2** is not a real, distinct hardware instruction.
  - It is a silicon alias for **SUBS XZR, X1, X2**. (Subtract **X2** from **X1**, throw the mathematical result into **XZR**, but keep the PSTATE updates).

Unconditional Branching

- In AArch64, you cannot write to the PC directly (e.g., **MOV PC, X0** is illegal). You must use Branch instructions.
- **B (Branch):**
  - Unconditionally shifts the PC to a new label.
  - Syntax: **B .loop**
- The instruction is strictly 32 bits wide. It cannot contain a 64-bit absolute RAM address.
- It stores a **PC-relative offset**.
- Maximum Range: **± 128 MB** from the current instruction.
  - *32 - 6 (opcode) + 2 (4 byte instructions) - 1 (sign bit) -> 2^{27} / 2^{20} = 128 MB*

Conditional Branching

- **B.cond** (Branch Conditionally):
  - Look at the PSTATE condition flags (N, Z, C, V) before deciding to alter the PC.
- Common Conditions:
  - **B.EQ** (Branch if Equal): Triggers if **Z == 1**.
  - **B.NE** (Branch if Not Equal): Triggers if **Z == 0**.
  - **B.LT** (Branch if Less Than): Evaluates **N** and **V** flags.
- Because the 32-bit instruction must now spend bits encoding the type of condition, the offset space shrinks. Maximum Range: **±1 MB**.
  - *32 - 9 (opcode) - 4 (condition code) + 2 (4 byte instruction) - 1 (sign bit) =* $2^{20}/2^{20}$ *= 1 MB*

Efficient Zero Checks (CBZ / CBNZ)

- The Standard Zero Check:
  - CMP X0, #0 (Compare with zero, overwrites PSTATE flags).
  - B.EQ .label (Read from PSTATE, branch).
- The AArch64 Solution:
  - CBZ X0, .label (Compare and Branch on Zero).
  - CBNZ X0, .label (Compare and Branch on Not Zero).
- Combines a zero-check and a branch into a single instruction (saving a CMP).
- Preserves ALU flags: Bypasses PSTATE entirely, allowing you to keep previous flags intact for later instructions.

Function Calls

- **BL** (Branch with Link):
  - Used to call a subroutine/function.
  - 1: Branches to the target address.
  - 2: Saves the address of the next instruction (PC + 4) into X30 (The Link Register).
- **RET** (Return):
  - Used at the end of a subroutine.
  - Copies the value in X30 back into the PC, returning execution to the caller.
- If Function A calls Function B, X30 is overwritten. Function A must push X30 to the stack before calling B.

Assembler Directives & Bare-Metal Pragmatics

Assembler Directives

- **What is a Directive?**
  - Commands that start with a period (.).
  - They do not execute on the CPU. They are instructions to the GNU Assembler (**as**) on how to construct the ELF file.
- **.section <name>**:
  - It applies to all subsequent lines of code or data, across all labels, until a new .section directive is explicitly declared.
  - e.g., .section .text (Executable code), .section .data (Initialized RAM), …

Assembler Directives

- **.global <symbol>:**
  - Makes a memory label visible to the Linker. Without this, functions are private to the file (like static in C).
- **.balign <bytes>:**
  - Forces the assembler to pad the binary with zeros until the next line of code or data lands on a mathematical multiple of <bytes>.
  - Crucial for preventing the hardware alignment faults we discussed earlier.

Allocating Physical Memory

- We don't have C types (int, long) in Assembly. We only allocate bytes.
- Data Size Directives:
  - **.byte**: Allocates 1 byte (8 bits).
  - **.word**: Allocates 4 bytes (32 bits).
  - **.quad**: Allocates 8 bytes (64 bits).
- Strings:
  - **.ascii "Hello"**: Allocates exactly 5 bytes.
  - **.asciz "Hello"**: Allocates 6 bytes (Appends a Null-terminator 0x00).

Example (An Array in RAM)

```
.section .data
.balign 8
my_array:
    .quad 10, 20, 30  // Allocates 24 bytes of contiguous physical RAM
```

We declare a section of data, we align it to an 8-byte boundary, we create a label my_array, and we lay down three 64-bit numbers. The assembler will embed the numbers 10, 20, and 30 sequentially into the binary. The label my_array simply becomes the memory address of the number 10.

Literal Pools and Pseudo-Instructions

- You cannot fit a 64-bit address (my_array) inside a 32-bit instruction.
- **MOV X0, #0x1234567890ABCDEF** won't compile.
- So we use this pseudo-instruction: **LDR X0, =my_array**
- What the Assembler does:
  - Creates a data section (a literal pool) near the code.
  - Drops the 64-bit address of my_array into that hidden pool.
  - Replaces the LDR = command with a PC-relative load (**LDR X0, [PC, #offset]**) to fetch the address out of the pool.

Remider: We don't have an OS!

- User-Space Assembly (Linux/macOS):
  - To print text or exit, you put a System Call Number in X8 and execute **SVC #0** (Supervisor Call).
  - The OS kernel intercepts it, does the complex hardware routing, and returns.
- Bare-Metal Assembly (Our Pi 5):
  - We are the Kernel. There is no OS to catch an SVC call.
  - Executing SVC #0 right now will cause an unhandled synchronous exception and halt the CPU.

Automating the Hardware Bridge

The Iteration Problem

- Every code change requires:
  - Moving the SD card to a PC.
  - Copying the new kernel8.img.
  - Moving the SD card back to the Pi 5.
  - Power cycling the board.
- The SWD Solution (RAM Injection):
  - We leave our Halt Stub on the SD card permanently.
  - We use GDB over the SWD cable to overwrite the Pi 5's RAM with our newly compiled binary.

Scripting the Debugger

- GDB can execute a text file of commands on startup to automate repetitive tasks. (using the **-x** flag)
- GDB only knows about software (breakpoints, registers, memory). It does not know how to physically reset a motherboard.
- **monitor** acts as a passthrough tunnel. It sends hardware-specific commands directly to the OpenOCD daemon.
- **monitor reset init**: Commands OpenOCD to trigger the physical reset line on the Pi 5 and instantly halt the CPU.

The load Command & ELF Metadata

- The **load** command reads the Executable and Linkable Format (.elf) file.
- It parses the internal Program Headers to find the Load Memory Addresses.
- Streams the binary sections over the SWD directly into the RAM.
- This is why we need the .elf file!
    - We feed GDB the .elf file so it knows exactly where in RAM to inject the code (0x80000 in our case) and where the debug symbols are.